ABI and Codegen

CS 1622

Jarrett Billingsley

Class Announcements

- how's the project?
- project 2 grades will be posted today!

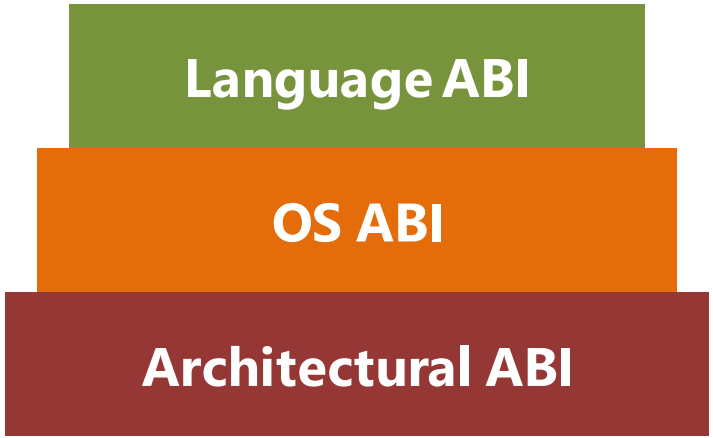
Application Binary Interfaces (ABIs)

Like API, but with a B instead of a P

- the backend converts the concept of "a program" into something the CPU can actually execute – **machine code.**
- an **Application Binary Interface** is a **set of rules** that sets the standards for how **machine code** programs look and work.
  - so, the ABI will guide how we actually generate the code.

to make things more interesting, "the" ABI is probably composed of multiple layers of rules.

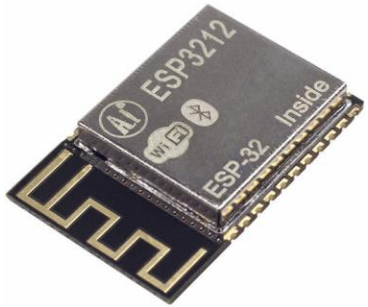
things can get very interesting at the **boundaries** between these layers…



The lowest layer: the Architectural ABI

- the designers of an ISA *may* define an ABI to go with it.
  - as the designers, they may have included special features for e.g. making function calls faster if you use those features as intended.
- these may be bare-bones, including rules on how to do **function calls** and **object file formats,** and not much else.

these are widely used on **embedded devices** where there is no OS – your program runs "on the bare metal."







But Operating Systems have opinions

- the goal of an operating system is to **control access to your computer's resources** and make the programs **share** them.
- so, the designers of OSes will define their **own** ABI which are much more extensive, including rules for:
  - how **function calls** work
    - this *may* be built on the architectural ABI, or it may be unique!
  - how **system calls** work, to ask the OS to do things for you
  - how **asynchronous messages** are delivered to running programs
  - how **executables** and **shared libraries** are packaged
  - how **memory** is laid out and where parts of your program exist
- **these** are the kinds of ABIs most compilers are generating code for, most of the time.

The highest layer: your language's own ABI

- HLLs can have features which are not accounted for by either the architectural or OS ABIs, so they have to make their own rules.
- this might include features like:

exceptions!

```
try {
  ...throw...
} catch(...) {
  ...
} finally {
  ...
}
```

dynamic method dispatch!

```
String s = obj.toString();
```

function closures!

```
auto f = [&]() {
  cout << "wheee!\n";
};
```

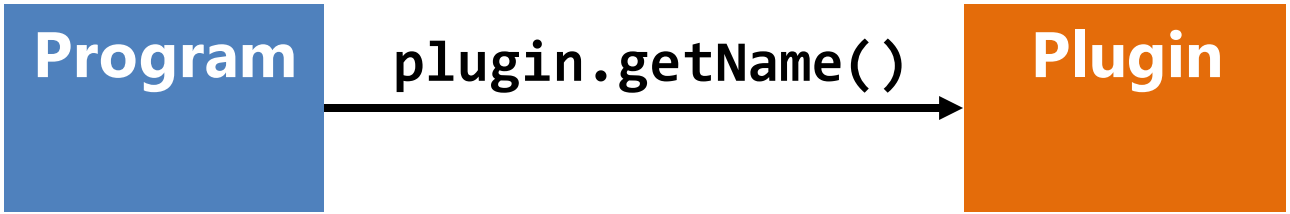
however, if you're not careful defining your language, **different compilers** for the **same language** can use different ABIs...

When ABIs collide

- let's say a language has **no standardized ABI for method calls.**
- then, different compilers may have their own **incompatible** ways of generating code for method calls, leading to really nasty situations.

this program was compiled with **Compiler A.**

you wrote a **plugin** in the *same language,* but you used **Compiler B.**



the program tries to **call a method** in your plugin, and… what happens?

who knows? it might silently fail, it might crash immediately, it might corrupt the stack or heap in more subtle ways, etc.

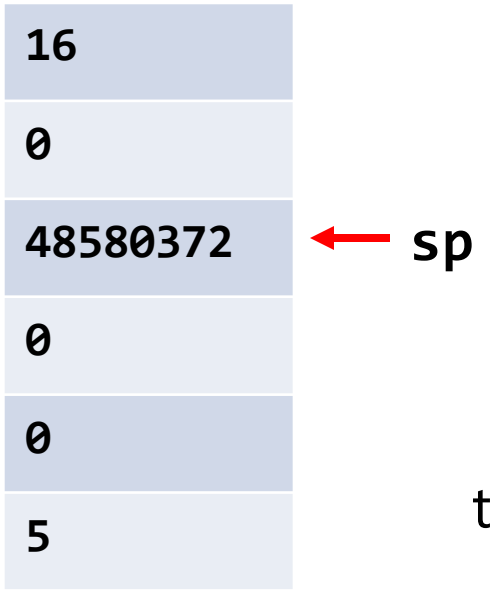
Break the rules, get a broken program.

- if **two pieces of code** use **different ABIs...**
- or if your compiler **generates code** that doesn't follow the ABI...
- **you lose all the nice abstraction guarantees of the HLL.**
- think back to when you were writing MIPS in 447.
  - *do you really want to deal with **those** kinds of bugs again?*
- C – or rather its ABI – has remained popular for decades. why?
  - because it's **super simple,** meaning just about every OS and compiler can follow it without much difficulty.
  - with it, you can hook pieces of machine code together **reliably.**
- but, its simplicity means **it doesn't support advanced features...**
  - so it's up to you, as the language designer or compiler writer, to decide how those features are implemented.
- so! what goes into an ABI, anyway?

Parts of an ABI

The call stack

- any ISA from the last 50 years or so will have provisions for a **stack.**



it'll have, at the very least, a register that points to the top of the stack… a *stack pointer,* if you will.

(well, maybe it has two registers. we'll get to that.)

it *might* have special instructions for pushing and popping things to this stack, or it might not.

```
push eax    pop ecx    call func    ret
```

the ABI defines things like: the **direction** the stack grows; the **size** of each item on the stack; stack **alignment** (if the stack pointer must move in multiples of some number); and so on.

You can't stack things up forever...

- statistically, most programs never need more than a few megabytes of stack space, and most use *far, far less* than that.
- since the OS makes all the programs share resources, it may only allocate something like **4-16 MB of memory** for your program.
  - this is allocated *before your program starts;* you don't have to do anything to get it.
- and what happens if you push more than that on the stack?
  - **stack overflow! yay!**
- so, *however* we implement our language's ABI, it will have to work **within this limit.**

The calling convention

- this is the set of rules used to **call** and **return from** functions.
- it defines things like:

how arguments are passed;

how the return address is calculated and where it's stored;

```
fn add(x: int, y: int): int {
  let sum = x + y;
  println_i(sum);
  return sum;
}
```

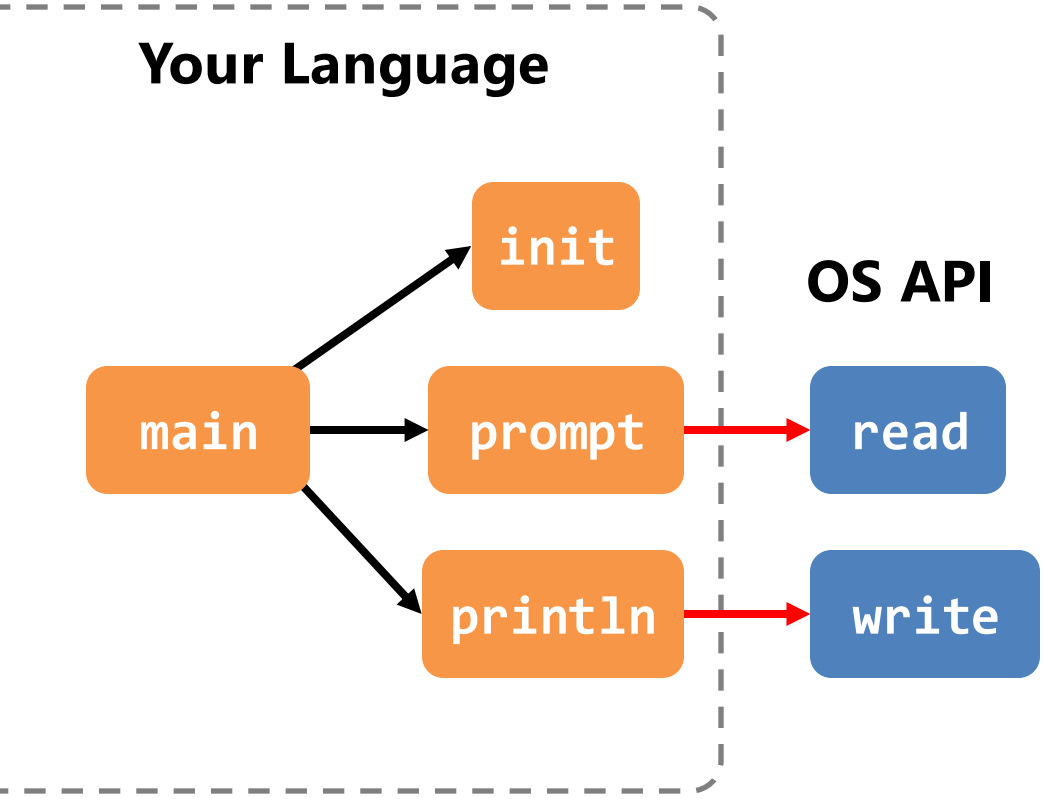
how values are returned;

which CPU registers must be saved, and which can be used freely.

s0 v0 ra f0 t0 a0 fp

Border conflicts

- *within* your language, you can use any calling convention you like.
  - as long as every function uses the **same rules,** it will work fine.
- it's at the **boundaries** where things get tricky.



functions from **outside your language** may use a **different ABI,** so your code generator must be very careful about it.

it has to know **what ABI** each function uses, so it can use the right rules when calling them.

or, we could punt it...

If it hurts, don't do it

- you could make it so **you can't directly call functions from other languages/which use different ABIs.**
- Java does this.
  - `public static native int someExternalFunction();`
  - this function is not written in Java.
  - instead, it's probably written in C++, and compiled into a plugin.
  - that plugin is then loaded into the JVM when your program runs.
- this provides a controlled boundary between "the safe Java world" and "the scary non-Java world," and lets Java programs call functions they would normally be unable to.
- **systems** programming languages will give you a way to deal with calling conventions yourself.
  - but Java isn't a systems language, and there's no shame in saying "hey, it's not my language's responsibility!"

Another kind of call, another calling convention

- **system calls** are how your program asks the OS to do things on the program's behalf: I/O, resource management, even *exiting*...
  - without syscalls, about all a program can do is make the CPU hot.
- the **ISA** defines the system call (syscall) mechanism...
  - which usually requires using a special instruction. (like `syscall`.)
  - the OS will also define how values are passed into/out of them.
- essentially this is **just another calling convention,** meaning we will run into the same issues as before.
  - if a language cannot use syscalls directly, it will have to rely on code written in some other language to do so.
- it feels like this **interface between our language and the OS** is going to have to be somewhat... substantial. so let's talk about it.

Standard and Runtime Libraries

std::

- you kind of take the standard library (stdlib) for granted.
  - there are these functions and classes that are just… there!
  - **`System.out.println()`** and **`ArrayList`** and **`Vec`** and so on.
  - but *someone* had to write them. and if you're making your own language, **that "someone" is going to be *you*.**
- standard libraries are out of the scope of this course, but…
  - the standard library *is* going to have to make calls to OS APIs to do things like input and output, *somehow*.
- and *all* code in your language will rely on some special hidden code to work properly at runtime.
  - and **the compiler** will be relying on that code as well!

core::

- there are likely many operations in your HLL that don't map neatly onto a few instructions in the target ISA.

is there a MIPS instruction for string concatenation?

```
"hi" + "bye"
```

how about for allocating objects on the heap?

```
new Cat()
```

checking array bounds?

```
A[1000000000]
```

throwing exceptions?

```
throw new OhMyGodWhyAreAllTheExceptionNamesSoLongException();
```

there are a lot of hidden operations going on in each of these cases, and it is the **runtime library** which implements them.

The language runtime

- a language's **runtime library** implements the features which would be difficult or impossible to codegen directly.
- unlike the standard library, this is tied more closely to the **compiler** and how the compiler has decided to implement those features.
  - so, different compilers – and even different versions of the *same* compiler – can have **different runtime libraries.**
  - ever install the many versions of Visual C++ Redistributable?
    - thaaaaat's what it iiiiiiiis
- the runtime may be written in a different language (like C or C++) from the source language that the compiler compiles.
  - in our case, Truss's tiny runtime library is written in MIPS. (so is its "standard library" of a few print functions, lol.)

The compiler's job

- during codegen, the compiler is essentially going to turn some pieces of code into **function calls** into the runtime library.
- if we take string concatenation as an example:

source code:

```
let msg = "hello, " + name;
```

codegen treats it as:

```
let msg = rt$concat("hello, ", name);
```

gets generated as:

```
addi sp, sp, -8
la   t0, STRLIT$3
sw   t0, 0(sp)
lw   t0, 0(fp)
sw   t0, 4(sp)
jal  rt$concat
sw   v0, -20(fp)
```

fp? what's that??

Truss's MIPS ABI

Kind of standard, kind of not

- Truss's MIPS ABI is *based on* the standard MIPS ABI, but is more limited in some ways in order to make the codegen simpler.
- we're generating code for a slightly weird target, MARS, which:
  - has no operating system
  - has no executable format, only *textual* assembly as input
- those aspects will *also* simplify our job!
- a large part of the ABI is the calling convention, but first, we have to specify…

Memory regions

- there are **three** regions of memory.

```
let g1 = 10;
let g2 = "world";
```

the .data segment holds the global variables and any string literals.

```
fn main() {
  println_i(g1);
  let msg = "hi, " + g2;
  println_s(msg);
}
```

the heap holds dynamically-allocated strings and structs (like `new Cat()`).

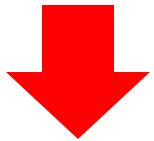
the stack holds the local variables.

the heap *could* be managed with a tracing garbage collector, if this were a real language. but our implementation won't.

How global variables are generated

- **int** and **bool** globals become **.word** and **.byte** variables, respectively.

```
let ivar = 10;
let bvar = true;
```

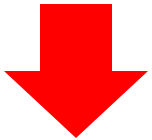


```
.data
ivar: .word 10
bvar: .byte 1
```

bools are represented with 1 for true and 0 for false.

strings are more subtle…

```
let svar = "hi";
```



```
.data
STRLIT$0: .asciiz "hi"
svar: .word STRLIT$0
```

every string literal in the program gets a uniquely-named **STRLIT** label; string *variables* just *point* to strings.

Runtime data representation

- to be thorough we have to define how *every* type in the language is represented at runtime. fortunately, Truss doesn't have many!
  - we already saw **int** and **bool** on the previous slide.
- **strings, function pointers,** and **struct references** are all *reference types,* meaning they are represented as a pointer, which is a **word.**
- the *data* of a **string** is defined as zero-terminated ASCII, which is kind of a terrible string representation but is exactly what MARS wants, so.
- the *data* of a struct is… well… we'll come back to structs. ;o

Function calls

- the **caller** has an easy time calling functions, but the protocol is a little different than you may remember from 447.
- rather than using those **a** registers, **all arguments are passed on the stack, in the same order they are written in the source.**

```
f(1, 2);
```



each stack slot is 4 bytes, so 2 arguments requires 8 bytes.

```
addi sp, sp, -8
li   t0, 1
sw   t0, 0(sp)
li   t0, 2
sw   t0, 4(sp)
jal  f
```

recall this means `0 + sp`.

the second argument is at `4 + sp`; the third at `8 + sp` etc.

After the call completes…

- return values come out in **v0,** like you learned before.
- assuming **glob** is a global variable…

```
glob = f(1);
```



```
addi sp, sp, -4
li   t0, 1
sw   t0, 0(sp)
jal  f
-----------------------
sw   v0, glob
```

notice: the caller **changes sp *before* calling,** but it does not change it *after*. we say the **callee "cleans the stack"** (as we'll see shortly)

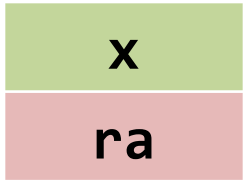
important: there is a line here. below this line, the contents of the **a**, **t**, and **v** registers **may be different** than they were before the **jal.**

however, the contents of the **s** registers **are guaranteed to be the same as before the jal.**

Stack Frames

- every function in the program needs a **stack frame:** a way of storing all the arguments, variables, and saved registers on the stack.
- let's look at a super simple function's stack frame first.

```
fn simple(x: int) {
  println_i(x);
}
```



there's the argument. it was pushed by the caller.

we're going to push **ra,** the return address register, so we won't lose our way back to the caller when we call **println_i**.

what about when we have **local variables?**

Local variables

- after pushing **ra**, we will then make space for the locals.

```
fn more_complex(x: int) {
  let y = x + 10;
  for i in 0, y {
    println_i(i);
  }
}
```



every local variable gets its own stack slot.

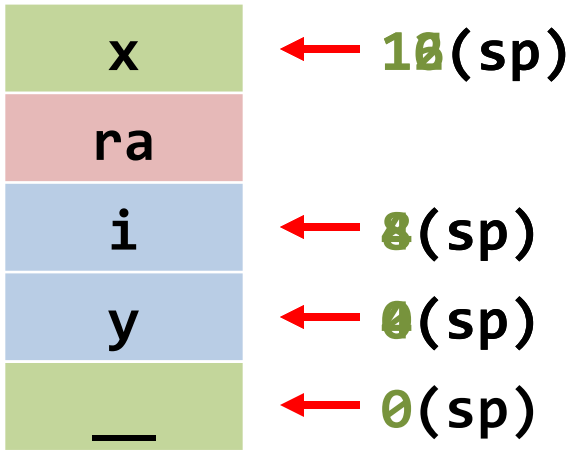
since the **stack pointer** is pointing at the lowest slot, we can access the locals and arguments with offsets from it.

but that's going to make things difficult for us when we want to actually call **println_i**...

Shifting sands (animated)

- let's say we're generating code for **`println_i(i)`**.

```
fn more_complex(x: int) {
  let y = x + 10;
  for i in 0, y {
    println_i(i);
  }
}
```



the **first thing we do** is make room for the argument.

```
addi sp, sp, -4
```

but **what does that do** to the offsets from sp??

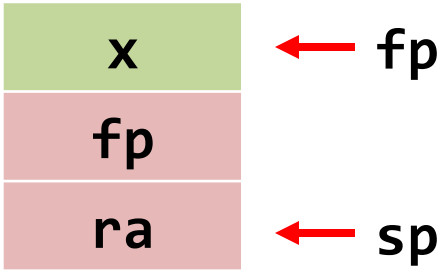
it's absolutely possible to keep track of the "virtual sp" and adjust the offsets accordingly. *but I don't waaaaanna*

what if we had a **second stack pointer,** one that **didn't move around?**

The Frame Pointer register

- MIPS has a second register for just this purpose, called **fp.**
- this will point sort of weirdly into the *middle* of the stack frame, giving us a **stationary reference point** for accessing locals.
- going back to the super simple function…

```
fn simple(x: int) {
  println_i(x);
}
```



we're going to push both **`fp`** *and* **`ra`!**

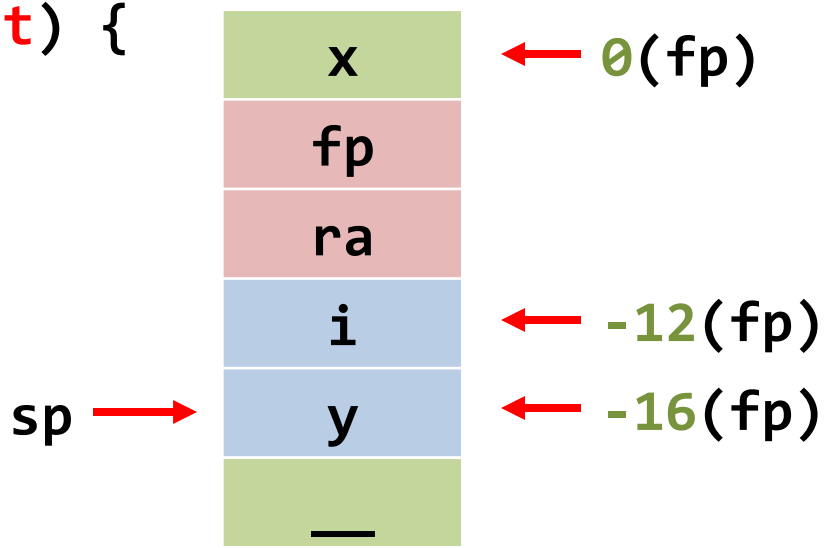
`fp` points at **the first argument,** and
`sp` still points at the top of the stack.

and what about locals?

The local problem, solved (animated)

- you aren't restricted to *positive* offsets from the stack/frame pointers!

```
fn more_complex(x: int) {
  let y = x + 10;
  for i in 0, y {
    println_i(i);
  }
}
```



now, even when **sp** moves around during the function, the offsets from **fp** never change.

Function Prologue

- the **function prologue** is the code that sets up the stack frame, essentially switching stack frames **from** the caller **to** the callee.
- it looks like this:

`sw fp, -4(sp)`	this pushes the *caller's* frame pointer, so we can get it back later.
`sw ra, -8(sp)`	this pushes *the callee's* return address.
`move fp, sp`	this sets up the *callee's* frame pointer, so it points to the first argument (if any).
`addi sp, sp, -n`	and finally, this sets up the *callee's* stack pointer, so we have room for the locals.

here, *n* is the number of bytes needed for all the locals, plus the two saved registers from above.

Function Epilogue

- finally, the **epilogue** does the opposite: it restores the stack and required registers to **the way they were when the function began.**

```
lw   ra, -8(fp)
```
this restores our return address.

```
lw   fp, -4(fp)
```
this restores the *caller's* frame pointer.

```
addi sp, sp, x
```
this **completely cleans the stack,** removing the locals, saved regs, *and* all arguments.

```
jr   ra
```
and this returns!

What about the juicy middles?

- it seems like a lot of buildup to only a handful of instructions, but...
  - this shit is important okay
  - we have to do this right or else our program falls apart!!
- but by setting up our stack frame neatly like this, it will make generating the code *inside* the functions a lot more straightforward.
  - and that's what we'll talk about next time!